

07

2시간

개발 업무 자동화 실습

테스트 생성

코드 리뷰

Git 자동화

리팩토링

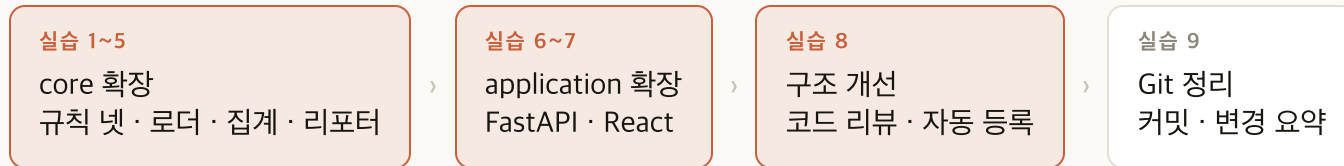
웹 애플리케이션

이 챕터에서 다루는 내용

주제	내용	실습
다건 규칙	여러 청구를 함께 보고 판정하는 규칙 세 개	1
테스트 생성	규정 문구부터 픽스처와 테스트까지 직접 작성	2 · 4
입출력 확장	Markdown 표를 읽는 로더와 표로 내는 리포터	3 · 5
집계	판정 결과를 부서별로 묶는 새 계산 축	4
웹 애플리케이션	FastAPI와 React를 진입점으로 추가	6
코드 리뷰 · 리팩토링	확장한 코드 검토와 규칙 등록 방식 교체	7
Git 작업 자동화	커밋 단위 분할과 메시지 작성	8

이 챕터는 새 도구를 배우는 자리가 아닙니다.
 챕터 4부터 쌓아 온 `CLAUDE.md` 와 Hook, Skill, Subagent를 그대로 쓰면서 앱을 확장합니다.

챕터 7의 작업 순서



- **core 확장 (실습 1~5):** 규칙 넷을 더해 경비 규정 항목 스물한 개를 모두 검사하고, Markdown 입출력과 부서별 집계를 붙입니다.
- **application 확장 (실습 6~7):** HTTP API와 React 화면을 진입점으로 추가합니다. API는 계획을 승인한 뒤에 맡기고, 화면은 자동 승인으로 맡깁니다.
- **구조 개선과 이력 정리 (실습 8~9):** 확장한 코드를 코드 리뷰로 점검하고 규칙 등록을 자동 방식으로 바꾼 뒤, 변경을 커밋 단위로 나눕니다.

개발 업무를 Agent에게 맡기고 결과를 확인합니다.

규칙 구현과 테스트 작성, 코드 리뷰, 리팩토링, 커밋까지 아홉 번의 실습이 모두 실무에서 하는 작업입니다. 앞 챕터에서 만든 하네스가 결과 검증을 대신합니다.

지금까지 갖춘 하네스

장치	종류	맡고 있는 일	만든 곳
CLAUDE.md	메모리	계층 구조와 코딩 컨벤션, 등록 규약	챕터 4
check-registration.py	Hook	등록이 빠진 채로 커밋되는 것을 차단	챕터 4
/rule-status	커스텀 명령	구현한 규칙과 남은 규칙 현황	챕터 5
/add-rule	Skill	규칙 구현 · 테스트 · 등록을 한 번에	챕터 5
rule-spec-check	Subagent	규정 문서와 구현 대조	챕터 5
settings.json	설정	승인 정책, 마스킹 Hook, 감사 로그	챕터 6

이 챕터의 요청은 짧습니다.
무엇을 어느 계층에 두고 어디에 등록할지는 하네스에 이미 적혀 있습니다. 요청에는 원하는 결과만 담습니다.

남은 규칙 네 개

	R017	R019	R020	R021
규칙	중복 청구	부서 월 한도 초과	1일 식비 중복	교육훈련비 사전승인
유형	다건	다건	다건	단건
규정 문구	docs/경비규정.md 에 있음			없음. 화면으로 제시
픽스처 · 테스트	픽스처 제공. 테스트는 Skill이 생성			둘 다 직접 작성
실습	실습 1			실습 2

규칙 스물한 개 중 R021 하나만 처음부터 끝까지 직접 만듭니다.
규정 문구를 옮겨 적는 단계부터 테스트 데이터를 설계하는 단계까지 전 과정을 거칩니다.

다건 규칙 세 개의 판정 기준

규칙	판정 조건
R017 중복 청구	사번, 사용일자, 금액, 가맹점이 모두 같은 건이 2건 이상이면 해당 건을 모두 반려
R019 부서 월 한도	사용월별로 사용일자 오름차순으로 금액을 누적해, 부서 한도를 넘긴 건부터 그달의 남은 건을 모두 반려
R020 1일 식비 중복	같은 사번이 같은 사용일자에 식비를 2건 이상 청구하면 해당 건을 모두 반려

- **판정에 필요한 다른 행:** 청구 한 건만 보아서는 중복인지 한도를 넘었는지 알 수 없습니다. `BatchRule` 을 상속하고 `check_all` 로 전체를 받습니다.
- **저장소에 있는 참고 구현:** 챕터 4에서 만든 R010 영수증번호 중복 사용이 같은 형태입니다.

실습 1 · 다건 규칙 세 개

- 1 요청: `/add-rule R017 R019 R020`
- 2 Skill이 규칙 구현과 테스트 작성, 두 곳 등록을 처리하는 동안 화면을 지켜봅니다
- 3 요청: `rule-spec-check` 로 R017, R019, R020 을 규정과 대조해 줘
- 4 `uv run pytest` 로 통과 건수를 확인합니다
- 5 `uv run settle samples\연구소.csv` 로 반려가 늘었는지 확인합니다

세 개를 한 번에 넘깁니다.

챕터 5에서 단건 일곱 개를 처리한 Skill이 다건 규칙에도 그대로 쓰입니다. Skill이 규정 항목 제목에 붙은 (다건)·(단건) 표시를 보고 상속할 베이스 클래스를 정합니다.

실습 1 실행 결과

uv run pytest

```
..... [100%]
22 passed in 0.03s
```

uv run settle samples\연구소.csv · 끝부분

```
EXP-2026-0396 임도연 2026-07-17 소모품비 26,000 반려 R017
EXP-2026-0397 임도연 2026-07-17 소모품비 26,000 반려 R017
EXP-2026-0398 한지훈 2026-07-20 도서구입비 38,000 반려 R017
EXP-2026-0399 한지훈 2026-07-20 도서구입비 38,000 반려 R017
EXP-2026-0400 임도연 2026-07-21 식비 21,000 반려 R020
EXP-2026-0401 임도연 2026-07-21 식비 24,000 반려 R020
전체 45건 · 통과 36 · 반려 9
```

다건 규칙은 반려된 건이 짝을 이루어 나옵니다. 한 건만 보아서는 반려된 이유를 알 수 없습니다.

R021 규정 원문

docs/경비규정.md 끝에 이어서 추가

R021 · 교육훈련비 사전승인 (단건)

계정과목이 교육훈련비이고 금액이 200,000원을 초과하는데
비고에 AP- 로 시작하는 사전승인 번호가 없으면 반려합니다.

> 20만원을 초과하는 교육훈련비는 사전승인 번호가 필요합니다

이 항목만 저장소에 없습니다.

화면의 문구를 규정 문서에 그대로 옮겨 적는 것이 실습 2의 첫 단계입니다. 항목 제목의 (단건) 표시로 상속할 베이스 클래스가 정해지고, 인용 문구가 위
반 메시지가 됩니다.

규칙 하나가 걸치는 다섯 지점



- **R001~R020에서 채워져 있던 두 지점:** 규정 문구가 문서에 있었고 픽스처가 저장소에 있었으므로, 만든 것은 구현과 테스트, 등록 세 가지입니다.
- **R021에서 직접 만드는 두 지점:** 규정 문구를 옮겨 적고 픽스처 데이터를 설계하는 일이 앞에 붙습니다.

`/add-rule` Skill의 3번 절차가 이 규칙에서는 성립하지 않습니다. 픽스처가 이미 있다고 적혀 있는데 `r021.csv` 는 없습니다. Skill을 그대로 호출하면 어떤 결과가 나오는지 확인한 뒤 대응 방식을 정하세요.

픽스처 작성 기준

픽스처는 규칙 하나를 검증하는 최소 데이터입니다. 아래 다섯 가지를 기준으로 행을 구성합니다.

기준	내용
대상 규칙 한정	다른 규칙의 위반이 섞이지 않도록 나머지 열을 조정
위반 2건 · 통과 2건	기대값 목록을 행 순서와 같게 구성
경계값 포함	200,000원은 통과, 200,001원부터 반려
예외 사례 포함	금액이 크더라도 비교에 AP- 로 시작하는 사전승인 번호가 있어 통과하는 건
헤더 일치	18개 열 전부. docs/데이터-스키마.md 의 순서를 따름

앞 챕터에서 만든 골든 세트 두 개(**base** , **set-2**)에는 R021 위반이 한 건도 없습니다. 규칙을 추가한 뒤 **base** 나 **set-2** 가 깨졌다면 픽스처가 아니라 구현이 규정과 다른 것입니다.

실습 2 · 규칙 R021

- 1 슬라이드 p9의 규정 문구를 `docs/경비규정.md` 끝에 추가합니다
- 2 요청: 경비규정에 R021을 추가했어. 규칙과 픽처, 테스트를 만들어 줘
- 3 만들어진 `tests/rules/fixtures/r021.csv` 를 열어 슬라이드 p11의 기준에 맞는지 확인합니다
- 4 요청: rule-spec-check 로 R021 을 규정과 대조해 줘
- 5 골든 세트 set-3 을 내려받아 `tests\golden\set-3` 에 놓습니다
- 6 `uv run pytest` 로 골든 세 세트가 모두 통과하는지 확인합니다
- 7 `uv run settle samples\개발2팀.csv` 로 사전승인 번호가 있는 건과 없는 건이 갈리는지 확인합니다

픽처를 읽어 보는 3번이 이 실습의 핵심입니다.

테스트 데이터가 규칙 하나만 겨냥하지 않으면 그 테스트는 다른 규칙이 추가될 때 함께 깨집니다.

실습 2 결과 파일

tests/rules/fixtures/r021.csv · 판정에 쓰이는 세 열만 발췌

claim_id	expense_type	amount	note
EXP-2026-0405	교육훈련비	350000	
EXP-2026-0406	교육훈련비	250000	
EXP-2026-0407	교육훈련비	200000	← 경계값
EXP-2026-0408	교육훈련비	480000	사전승인 AP-2026-0117

tests/rules/test_r021.py

```
from core.domain.rules.r021_training_preapproval import R021

def test_r021(settle):
    result = settle([R021()], "r021.csv")
    codes = [[v.code for v in j.violations] for j in result.judgements]
    assert codes == [["R021"], ["R021"], [], []]
```

실습 2 실행 결과

uv run pytest · 골든까지 그대로 통과

```
..... [100%]
24 passed in 0.03s
```

uv run settle samples\개발2팀.csv · 교육훈련비 건만 발체

EXP-2026-0258	최수아	2026-07-20	교육훈련비	180,000	통과
EXP-2026-0266	최수아	2026-06-24	교육훈련비	450,000	반려 R021
EXP-2026-0267	오세훈	2026-06-25	교육훈련비	260,000	통과

전체 45건 · 통과 37 · 반려 8
R021 교육훈련비 사전승인 1건

마지막 두 건에서 규칙의 동작이 드러납니다. 통과한 26만원 건의 비교에 AP- 로 시작하는 사전승인 번호가 들어 있습니다.

입력 형식 추가

이미 갖춰진 부분

파일 열기	경로 확인과 읽기
인코딩	<code>utf-8-sig</code>
형식 판별	<code>supports</code> 호출
순서 보존	원본 행 순서 유지
CsvLoader	같은 자리의 구현체. 본보기로 삼음

새로 쓰는 부분

- `CsvLoader` 와 같은 모양의 `MarkdownLoader` 추가
- `supports` : 확장자가 `.md` 인지 판별
- `parse` : 표 텍스트를 `Expense` 목록으로 변환
- 진입점의 `--in md` 옵션과 조립부의 형식 대응표 추가

파일을 여는 코드를 새로 썼다면 구조를 잘못 읽은 것입니다.

형식과 무관한 일은 중간 추상 클래스가 이미 처리하고 있습니다. 구체 로더는 텍스트를 해석하는 부분만 채웁니다.

실습 3 · Markdown 로더

- 1 요청: `samples\영업1팀.csv` 와 같은 내용을 담은 Markdown 표 파일을 만들어 줘
- 2 요청: `CsvLoader` 와 같은 자리에 Markdown 표를 읽는 `MarkdownLoader` 를 추가해 줘. CLI 실행 시 `--in md` 로 선택되게 해 줘
- 3 요청: 같은 내용을 CSV와 Markdown으로 각각 읽었을 때 판정이 같은지 확인하는 테스트를 만들어 줘
- 4 `uv run pytest` 로 기존 CSV 테스트가 그대로 통과하는지 확인합니다
- 5 `uv run settle samples\영업1팀.md --in md` 를 실행해 CSV 결과와 비교합니다

완료 기준은 세 가지입니다.

내용이 같은 두 파일이 동일한 판정을 내는 것, 파일을 여는 코드를 새로 쓰지 않은 것, 기존 CSV 테스트가 그대로 통과하는 것입니다.

실습 3 실행 결과

```
uv run settle samples\영업1팀.md --in md
```

```
전체 51건 · 통과 36 · 반려 15  
R012 접대비 1인 한도 12건  
R019 부서 월 한도 초과 2건  
R021 교육훈련비 사전승인 1건
```

```
uv run settle samples\영업1팀.csv
```

```
전체 51건 · 통과 36 · 반려 15  
R012 접대비 1인 한도 12건  
R019 부서 월 한도 초과 2건  
R021 교육훈련비 사전승인 1건
```

두 형식의 판정이 같습니다. 새로 쓴 것은 표 텍스트를 해석하는 `parse` 하나입니다.

규칙과 집계의 확장 방식

집계는 판정이 끝난 결과를 부서 같은 기준으로 묶어 표로 만드는 계산입니다. 규칙이 청구를 한 건씩 판정하는 것과 달리, 집계는 판정 결과 전체를 대상으로 합니다. 아래 표는 규칙과 집계를 하나씩 추가할 때의 수정 지점을 비교합니다.

	규칙	집계
추가 파일	규칙 파일 1개	집계 파일 1개
연동 수정 지점	<code>container.py</code> 2개: 앱 · 테스트	<code>container.py</code> 1개: 앱
출력 계층	수정 불필요: 위반 객체가 규칙 이름을 함께 전달	수정이 필요 없는 구조 설계가 실습 과제

출력이 집계 종류를 모르게 만들면 집계를 추가해도 출력을 고칠 일이 없습니다.

리포터는 집계 목록을 순서대로 출력할 뿐이므로, 집계 추가는 목록에 항목 하나를 더하는 작업으로 끝납니다.

부서별 집계 표 명세

실습 4의 요청에 붙여 넣을 명세

부서별 집계 표의 열 (순서대로)

부서: 청구가 한 건이라도 있는 부서

청구: 건수

통과: 통과 건수

반려: 반려 건수

반려율: $\text{반려} \div \text{청구} \times 100$, 소수점 첫째 자리까지 반올림

청구금액: 금액 합계

반려금액: 반려 건의 금액 합계

이 명세를 복사해 실습 4의 요청에 붙여 넣습니다.

명세 없이 요청하면 모델이 열 구성과 반올림 기준을 임의로 정하고, 이후 슬라이드의 실행 결과 화면과 값이 달라집니다.

집계에서 판단이 필요한 지점

- **금액이 비었거나 음수인 건:** R002 위반으로 이미 반려된 건입니다. 금액 합계에 어떻게 반영할지 정합니다.
- **반올림이 일어나는 지점:** 계산 계층에서 한 번만 합니다. 표현 계층이 다시 반올림하면 터미널과 웹의 숫자가 달라집니다.
- **청구가 없는 부서:** 표에 넣지 않습니다. 판정 결과에 없는 부서를 어디선가 가져와야 한다면 계층을 넘은 것입니다.

요청에 함께 붙여 넣을 세 줄

집계 처리 기준

1. 금액이 비어 있는 건은 합계에서 0으로 본다. 반려금액에는 그 건의 금액을 그대로 넣는다.
2. 반올림은 계산 계층에서 한 번만 한다. 표현 계층은 다시 계산하지 않는다.
3. 청구가 한 건도 없는 부서는 표에 넣지 않는다. 반려율 내림차순으로 정렬한다.

세 항목을 적지 않으면 모델이 임의로 정합니다.

어느 쪽으로 정해도 결과는 그럴듯해 보이므로, 정한 내용을 요청에 함께 실어야 결과가 같은 형태로 나옵니다.

실습 4 · 부서별 집계

- 1 요청: 판정 결과를 부서별로 묶는 집계를 추가해 줘. 추상 클래스는 `Aggregator`, 구현은 `DepartmentAggregator`, 결과 표는 `AggregationTable` 로 만들고, `SettlementResult` 의 `aggregations` 에 담아 줘 (앞 슬라이드의 집계 명세와 집계 처리 기준 세 줄을 함께 붙여 넣어 전달)
- 2 요청: `ConsoleReporter` 가 집계 종류에 의존하지 않고 `aggregations` 를 순서대로 표로 출력하게 해 줘
- 3 요청: 파일을 읽지 말고 판정 객체를 테스트 코드에서 직접 구성해 넘기는 집계 테스트를 만들어 줘
- 4 `uv run pytest` 로 골든이 그대로 통과하는지 확인합니다
- 5 `uv run settle tests\golden\base\input.csv` 로 다섯 부서가 한 표에 나오는지 확인합니다

골든 테스트는 `judgements` 만 비교합니다.

결과 객체에 집계가 추가되어도 기대 출력은 달라지지 않습니다. 여기서 골든이 깨졌다면 집계가 판정을 바꾼 것입니다.

같은 절차를 반복하게 되면 파일로 옮깁니다.

집계를 하나 더 붙일 때마다 이 요청을 다시 쓰게 됩니다. `CLAUDE.md` 나 Skill에 절차를 적어 두면 다음부터는 집계 이름만 넘기면 됩니다.

실습 4 실행 결과

uv run settle tests\golden\base\input.csv · 다섯 부서가 섞인 입력

[부서별 정산 요약]

부서	청구	통과	반려	반려율	청구금액	반려금액
개발2팀	19	12	7	36.8	1,086,000	252,000
마케팅팀	19	12	7	36.8	1,529,000	545,000
연구소	19	12	7	36.8	1,772,000	452,000
영업1팀	18	12	6	33.3	1,429,000	259,000
경영지원팀	16	12	4	25.0	775,000	400,000

집계 표가 요약 뒤에 붙었지만 출력 코드에서 늘어난 것은 표를 그리는 부분뿐입니다.

리포터는 `result.aggregations` 를 순서대로 출력할 뿐, 어떤 집계가 들어 있는지 알지 못합니다.

출력 형식 추가

- 리포터가 돌려주는 것: 문자열 하나입니다. 화면에 출력하거나 파일로 저장하지 않습니다. 지금은 진입점이 받아서 출력합니다.
- 담기는 내용: 터미널 출력과 같은 건별 표와 요약에, 실습 4에서 만든 집계 표가 함께 들어갑니다.
- 정해야 할 것: Markdown 결과를 파일로 저장하는 책임을 어디에 둘지 정합니다.

입력과 출력이 비대칭입니다.

입력 쪽은 `FileLoader` 가 파일 열기와 인코딩까지 맡고 있는데, 출력 쪽 리포터는 문자열만 돌려줍니다. 비대칭을 그대로 둘지 대칭으로 맞추지 정하고 근거를 남기세요.

실습 5 · Markdown 리포트

요청 · 한 번에 전달

결과를 Markdown 으로 내보내는 기능을 추가해 줘.

1. Markdown 표로 출력하는 MarkdownReporter 를 추가하고, CLI 실행 시 `--out md` 로 선택되게 함
2. 건별 표 다음으로 부서별 집계 표도 넣음
3. 파일로 저장하는 것은 리포터가 아니라 CLI가 맡음. 리포터는 문자열만 돌려주고, CLI 가 `--save` 로 경로를 받아 저장. 상위 폴더가 없으면 만들

- 1 위 요청을 그대로 전달
- 2 **요청:** Markdown 출력에 판정 표와 집계 표의 헤더 행이 모두 들어 있는지 확인하는 테스트를 만들어 줘
- 3 `uv run settle samples\마케팅팀.csv --out md --save outputs\마케팅팀.md` 로 파일이 생겼는지 확인합니다

출력 형식 테스트는 구조만 확인합니다.

문자열 전체를 비교하면 서식이 조금만 바뀌어도 깨집니다. 표의 헤더 행이 있는지 보는 것으로 충분합니다.

실습 5 실행 결과

uv run settle samples\마케팅팀.csv --out md · 앞부분과 끝부분

판정

청구ID	성명	사용일자	계정과목	금액	판정	위반
---	---	---	---	---	---	---
EXP-2026-0268	강예린	2026-05-01	접대비	190,000	통과	

부서별 정산 요약

부서	청구	통과	반려	반려율	청구금액	반려금액
---	---	---	---	---	---	---
마케팅팀	44	36	8	18.2	4,336,500	942,000

집계 표가 새 리포터에도 그대로 나옵니다.

실습 4에서 만든 집계를 다시 구현하지 않았습니다. 리포터는 결과 객체의 aggregations 를 형식에 맞게 옮길 뿐입니다.

웹 애플리케이션으로 확장

application 아래에 cli/ 와 나란히 배치

```
src/
├── core/           그대로
└── application/
    ├── container.py
    ├── cli/
    └── web/        ← 새로
```

- **거치지 않는 출력 계층**: 웹은 판정 결과를 직렬화해 응답으로 돌려주므로 리포터를 쓰지 않습니다.
- **진입점마다 다른 조립**: CLI와 웹이 따로 실행되므로 각자 필요한 것만 조립합니다.
- **한 줄도 바뀌지 않는 core**: 바뀌었다면 경계를 넘은 것입니다.

실습 1부터 5까지와 방향이 반대입니다.

앞에서는 core 를 다섯 번 확장하는 동안 application 이 그대로였습니다. 이번에는 application 을 확장하고 core 를 그대로 둡니다.

기술 스택과 제약 조건

항목	내용
API 서버	FastAPI + Pydantic. <code>uv add fastapi uvicorn python-multipart</code>
화면	React + TypeScript. Vite로 생성
바인딩 주소	<code>127.0.0.1</code> 주의 <code>0.0.0.0</code> 으로 바인딩하면 Windows Defender 승인 창이 나타납니다
네트워크	PyPI · npm registry 접근 필요
API 명세	API 명세 문서 (Swagger UI) . 엔드포인트와 요청 형식, 응답 필드, 오류 응답을 규정
기존 테스트	웹 인터페이스 추가 후에도 <code>core</code> 중심 테스트 통과 건수 유지

패키지 설치가 막히면 화면 없이 API까지만 진행합니다. 백엔드만 만들어도 `core` 가 진입점에 묶여 있지 않다는 것을 확인할 수 있습니다.

plan 모드의 계획 승인

- **진입 방법:** `Shift+Tab` 으로 **plan** 모드로 바꿉니다. 챕터 3에서 본 권한 모드 넷 중 하나입니다. 파일을 쓰지 않고 무엇을 할지만 제시합니다.

계획에서 확인할 것	반려하는 경우
수정하는 파일 목록	<code>src/core</code> 가 들어 있음
만드는 엔드포인트	API 명세에 없는 경로가 있음
설치하는 패키지	명세에 필요 없는 의존성을 추가함

승인하기 전에는 파일이 하나도 바뀌지 않습니다.

반려하면 계획만 버리고 요청을 고쳐 다시 냅니다. 승인한 뒤부터 실행이 시작됩니다.

실습 6 · FastAPI 서버

- 1 **API 명세 문서** 를 열어 요청 형식과 응답 필드를 확인합니다
- 2 `Shift+Tab` 으로 **plan** 모드 전환
- 3 **요청:** 청구 파일을 받아 요약과 판정, 집계를 JSON으로 돌려주는 API를 `src/application/web/` 에 FastAPI로 추가해 줘. 명세는 `https://hagyutae.github.io/expense-settlement-lab/openapi.json` 을 그대로 따르고, `127.0.0.1:8000` 에서 실행되게 해. `core` 는 고치지 마
- 4 계획에 `src/core` 파일이 없는지 확인하고 승인
- 5 `127.0.0.1:8000` 에 서버를 실행하고 `http://127.0.0.1:8000/docs` 가 명세와 같은지 확인합니다
- 6 `uv run pytest` 로 기존 테스트가 그대로 통과하는지 확인합니다

FastAPI 는 Python으로 HTTP API를 만드는 프레임워크입니다.

`core` 를 고치지 않았는지는 계획에서 확인합니다.
진입점을 하나 더 추가하는 동안 `core` 가 그대로였다면 계층 경계가 실제로 작동한 것입니다.

실습 6 실행 결과

API 응답 · 판정 한 건

```
{
  "claim_id": "EXP-2026-0266", "department": "개발2팀",
  "expense_type": "교육훈련비", "amount": 450000,
  "is_passed": false,
  "violations": [{ "code": "R021", "name": "교육훈련비 사전승인" }]
}
```

기존 테스트

```
> uv run pytest          29 passed in 0.05s
```

진입점이 하나 늘었는데 **core** 는 그대로입니다. 웹은 판정을 직렬화해 응답으로 돌려줄 뿐이므로 출력 계층도 거치지 않습니다.

화면 구성

● ● ● http://127.0.0.1:5173

경비 정산

1 파일 선택

청구 파일 선택

2 요약

전체 45건 · 통과 37 · 반려 8

3 집계 표

부서별 정산 요약

부서	청구	통과	반려	반려율	청구금액	반려금액
개발2팀	45	37	8	17.8	3,333,500	786,000

4 판정 표

청구ID	성명	사용일자	계정과목	금액	판정	위반
EXP-2026-0249	최수아	2026-07-07	교육훈련비	190,000	통과	
EXP-2026-0266	최수아	2026-06-24	교육훈련비	450,000	반려	R021
EXP-2026-0267	오세훈	2026-06-25	교육훈련비	260,000	통과	

화면과 응답의 대응

영역 넷은 POST /api/settle 응답에서 나옵니다. 화면은 계산하지 않고 받은 값을 그리기만 합니다.

1 파일 선택

고른 파일을 multipart/form-data 의 file 로 보냅니다. .csv 와 .md 를 받습니다.

2 요약

summary 의 total passed rejected 를 그대로 씁니다. 화면이 세지 않습니다.

3 집계 표

aggregations[] 를 순서대로 그림니다. name 이 제목, columns 가 머리글, rows 가 행입니다. 어떤 집계가 오는지 화면은 알지 못합니다.

4 판정 표

judgements[] 의 일곱 필드를 그대로 씁니다. violations[] 는 code 만 모아 한 칸에 넣습니다.

집계 표는 응답을 따라 늘어납니다.
집계를 하나 더 추가해도 화면 코드를 고치지 않습니다. 표 하나를 그리는 코드가 배열을 도는 형태여야 합니다.

실습 7 · React 화면

- 1 `Shift+Tab` 으로 **auto** 모드 전환
- 2 요청: 그 API를 사용하는 React 화면을 저장소 루트의 `ui/` 에 Vite로 만들어 줘. `127.0.0.1:5173` 에서 실행되게 하고, 화면 구성은 <https://hagyutae.github.io/expense-settlement-lab/ui-mockup.png> 를 참고해 줘
- 3 승인을 묻지 않고 진행하는 동안 만들어지는 파일을 지켜봅니다
- 4 `npm run dev` 로 화면을 열어 샘플 파일을 올려 봅니다

React 는 화면을 구성하는 JavaScript 라이브러리, **Vite** 는 React 프로젝트를 생성하고 개발 서버를 실행하는 도구입니다.

앞 실습과 반대로 말합니다.

서버는 계획을 보고 승인했지만, 화면은 승인 없이 끝까지 말합니다. 결과가 화면에 그대로 드러나고 `core` 와 API를 고치지 않는 작업이므로 이 방식이 성립합니다.

실습 7 실행 결과

ui/ · 자동 승인으로 만들어진 구성

```
ui/
├─ index.html      package.json  vite.config.ts
└─ src/  main.tsx  App.tsx (업로드와 네 영역)  App.css
```

PowerShell · npm run dev

```
VITE v8.2.1  ready in 753 ms
→ Local:   http://127.0.0.1:5173/
```

화면이 API를 호출할 뿐 `src/` 와 섞이지 않습니다. 호출 주소는 `App.tsx` 한 곳에 있고, 판정 로직은 서버에 그대로 남아 있습니다.

Claude Code가 `ui/` 안에서 명령을 실행해도 Hook은 그대로 동작합니다.

Bash 작업 디렉터리는 명령 사이에 유지되므로, 상대 경로로 등록했다면 `ui/` 로 옮겨간 뒤 스크립트를 찾지 못합니다. 챕터 4에서 `${CLAUDE_PROJECT_DIR}` 기준으로 등록한 이유입니다.

코드 리뷰를 맡기는 두 경로

	/code-review	직접 정의한 Subagent
준비	기본 제공. 바로 호출	<code>.claude/agents/</code> 에 직접 작성
대상	현재 브랜치의 변경 전체	정의할 때 지정
기준	범용 결함. 버그와 정확성	프로젝트 고유 규약

두 경로 모두 별도 컨텍스트에서 실행되므로 앞선 대화의 판단에 영향을 받지 않습니다.

두 경로는 기준을 누가 정하느냐에서 갈립니다. 실습 8에서는 확장한 코드 전체를 `/code-review` 에 맡기고, 규정 대조는 챕터 5에서 만든 `rule-spec-check` 가 이어서 담당합니다. 등록 누락처럼 Hook이 이미 차단하는 항목은 어느 쪽에도 맡기지 않습니다.

/code-review 호출 형식

```
/code-review [effort] [ultra] [--fix|--comment|--post] [대상]
```

자리	쓸 수 있는 값	기본값
effort	low medium high xhigh max	직전에 입력한 레벨. 입력한 적이 없으면 세션의 현재 effort
ultra	ultra	없음. 세션 안에서 검토
플래그	--fix --comment --post	없음. 결과만 보고하고 파일을 고치거나 게시하지 않음
대상	파일 경로, 브랜치, PR 번호, main...feature	현재 브랜치가 upstream보다 앞선 커밋과 커밋하지 않은 변경

번들 Skill이며 별도 컨텍스트에서 실행됩니다.

앞선 대화의 판단이 넘어가지 않으므로, 무엇을 볼지는 호출할 때 함께 전달해야 합니다.

low 와 medium 은 확신이 높은 지적만 보고합니다.

범위를 넓히려면 high 이상을 지정합니다.

확장한 코드의 리뷰 기준

실습 8의 요청에 붙여 넣을 기준

다음 기준으로 확장한 코드를 검토해 줘
Markdown 로더: 파일 열기와 인코딩을 다시 쓰지 않았는가
집계: 반올림이 한 곳에서만 일어나는가, 출력이 집계 종류를 아는가
Markdown 리포터: 파일 저장 책임이 정해졌는가
웹: core 를 고치지 않았는가, 조립을 진입점이 부르는가
규칙 넷: 판정 조건과 위반 문구가 규정과 같은가

이 기준을 복사해 실습 8의 요청에 붙여 넣습니다.

기준 없이 리뷰를 요청하면 변수 이름과 서식 같은 지적이 앞에 옵니다. 무엇을 볼지 함께 전달하면 설계 문제가 먼저 드러납니다.

명시적 등록 방식의 현재 구조

src/application/container.py

```
from ...r001_required_fields import R001
from ...r002_amount_valid import R002
... 스물한 줄 ...

def build_service():
    return RuleBasedSettlementService([
        R001(), R002(), R003(), ...
    ])
```

- 두 파일에 있는 같은 나열: tests/container.py 에도 스물한 개가 그대로 있습니다.
- 두 곳의 나열을 지키려고 만든 장치: 챕터 4의 등록 확인 Hook과 챕터 5의 /add-rule Skill이 나열을 빠뜨리는 것을 막아 왔습니다.
- 구조를 바꾸면 사라지는 일: 규칙 파일을 추가하는 것만으로 양쪽에 반영되면 그 장치들이 하던 일이 없어집니다.

도구로 실수를 막는 단계에서 구조로 실수를 없애는 단계로 넘어갑니다.

docs/요구사항.md 의 '명시적으로 나열해 등록한다'를 걷어내는 것이 실습 8의 과제입니다.

등록 방식 변경 시 함께 고칠 곳

파일	고칠 내용
<code>docs/요구사항.md</code>	‘명시적으로 나열해 등록한다’ 요구 사항
<code>CLAUDE.md</code>	두 곳 모두에 등록하라는 코딩 컨벤션
<code>.claude/skills/add-rule</code>	등록을 지시하는 4번 절차와 점검 스크립트
<code>.claude/hooks/</code>	커밋 직전에 등록을 검사하는 Hook

구조를 바꾸는 일은 코드에서 끝나지 않습니다.

그 구조를 전제로 쓴 문서와 하네스가 함께 어긋납니다. 무엇을 고쳐야 하는지 찾는 것이 실습 8의 뒷부분입니다.

고치지 않은 하네스는 틀린 지시를 계속 내보냅니다. 등록하라는 절차가 남아 있으면 다음 규칙을 추가할 때 Skill이 이미 건너낸 나열을 다시 만들어 넣습니다.

실습 8 · 코드 리뷰와 등록 방식 교체

- 1 요청: `/code-review` (슬라이드 p36의 기준을 복사해 함께 전달)
- 2 지적 목록을 읽고 지금 고칠 것과 남길 것을 나눕니다
- 3 요청: 규칙 파일을 추가하는 것만으로 앱과 테스트 양쪽에 등록되게 바꿔 줘. 규칙 폴더에 `registry.py` 를 만들고 `@rule` 데코레이터와 `load_rules()` 를 두는 방식으로 해 줘
- 4 `uv run pytest` 로 골든이 그대로 통과하는지 확인합니다
- 5 요청: 등록 방식이 바뀌었어. 요구사항과 CLAUDE.md, Skill, Hook 에서 이전 방식을 설명하는 부분을 찾아 줘
- 6 찾은 목록을 확인합니다
- 7 요청: 찾은 곳을 모두 새 등록 방식에 맞게 고쳐 줘
- 8 `/rule-status` 로 규칙 현황이 그대로 나오는지 확인합니다
- 9 규칙 파일 하나를 임시로 놓고 등록 없이 반영되는지 확인한 뒤 지웁니다

출력이 달라지지 않는 변경입니다.

골든이 깨졌다면 규칙이 빠졌거나 두 번 등록된 것입니다. 안전망이 있어서 구조를 바꿀 수 있습니다.

등록 코드의 변화

바꾸기 전 · src/application/container.py

```
from ...r001_required_fields import R001    ← 스물한 줄
def build_service():
    return RuleBasedSettlementService([
        R001(), R002(), ... R021(),          ← 스물한 개
    ], ...)
```

바꾼 뒤

```
from core.domain.rules.registry import load_rules
def build_service():
    return RuleBasedSettlementService(load_rules(), ...)
```

등록을 빠뜨릴 자리가 사라집니다. tests/container.py 도 같은 모양이 됩니다.

실습 8 실행 결과

규칙 파일 하나를 더 놓고 아무것도 등록하지 않은 상태

```
> uv run python -c "from core.domain.rules.registry import load_rules; print(len(load_rules()))"
22
```

골든은 그대로 통과

```
> uv run pytest
29 passed in 0.05s
```

파일을 추가한 것만으로 규칙이 스물두 개가 됐습니다.

출력이 달라지지 않았으므로 골든 세 세트가 그대로 통과합니다. 규칙이 하나라도 빠지면 그 규칙을 검사하는 세트가 깨집니다.

Git 작업을 맡기는 범위

작업	맡기는 내용
커밋 단위 나누기	변경을 주제별로 묶어 스테이징
커밋 메시지	스테이징된 변경을 읽고 제목과 본문 작성
변경 요약	브랜치 전체의 차이를 읽고 PR 본문 초안 작성
이력 조회	<code>git log</code> 를 읽고 언제 무엇이 바뀌었는지 답변

읽을 양이 많고 판단이 적은 작업을 맡깁니다.

위 네 가지는 모두 diff와 로그를 끝까지 읽어야 하는 작업이지만, 읽고 나면 답이 거의 정해집니다. 직접 하면 지루하고 빠뜨리기 쉬운 일입니다. 무엇을 한 커밋으로 볼지처럼 의도가 개입하는 결정은 사람이 합니다.

사람이 판단하는 지점

- **푸시 없이 커밋까지:** 챕터 6에서 `git push` 를 `deny` 에 넣었고, 실습 저장소는 읽기 전용이므로 푸시 권한도 없습니다.
- **커밋 전 승인 절차:** 챕터 6에서 `git commit` 을 `ask` 에 넣어 두었으므로 승인 화면에 명령과 메시지가 먼저 표시됩니다. 메시지를 읽고 진행 여부를 정합니다.
- **파일로 남기는 PR 본문:** 원격에 올리지 않고 초안만 만들어 둡니다.
- **되돌릴 수 없는 명령:** `git reset --hard` 처럼 이력을 지우는 명령은 `deny` 나 Hook으로 막아 둘 수 있습니다.

되돌릴 수 없는 명령은 Hook으로 막습니다.

`git reset --hard` 와 `git push --force` 는 실행되고 나면 이전 상태로 되돌릴 수 없습니다. 승인 화면에서 사람이 매번 걸러 내는 대신 `PreToolUse` Hook으로 차단해 두면 놓쳐도 실행되지 않습니다.

실습 9 · Git 작업 정리

- 1 **요청:** 지금 작업 폴더의 변경을 주제별로 나눠서 커밋 단위를 제안해 줘
- 2 제안을 읽고 순서를 정합니다
- 3 **요청:** 첫 묶음을 스테이징하고 커밋해 줘. 메시지에는 무엇이 왜 바뀌었는지 적어 줘
- 4 승인 화면에서 메시지를 확인하고 승인합니다. 남은 묶음도 같은 방식으로 진행합니다
- 5 **요청:** 이 브랜치 전체의 변경을 PR 본문으로 정리해 `outputs/pr.md` 에 남겨 줘
- 6 `git log --oneline` 으로 이력을 확인합니다

제안을 검토한 뒤 진행합니다.

커밋 단위 구분은 대체로 타당합니다. 다만 팀이 정한 규칙이나 직접 작업하며 잡아 둔 흐름과 어긋날 수 있습니다. 이력에 남는 결과의 책임은 사람에게 있으므로 그대로 받아들이지 않고 2번에서 확인합니다.

실습 9 결과 · 변경 요약

브랜치 전체의 차이를 읽고 만든 PR 본문 초안

```
# 경비 정산 시스템 확장

## 변경 내용
| 계층 | 내용 |
|---|---|
| 규칙 | R017 · R019 · R020 · R021 |
| 입력 · 출력 | MarkdownLoader, MarkdownReporter |
| 처리 | Aggregator 와 DepartmentAggregator |
| 진입점 · 구조 | HTTP API 와 React 화면, load_rules() |

## 확인한 것
- uv run pytest 29 passed · 골든 세 세트 그대로 통과
```

초안을 검토한 뒤 형식은 Skill로 고정합니다.

빠진 맥락이 없는지 읽어 보고 덧붙입니다. 팀에 PR 템플릿이 있다면 요청마다 형식을 적는 대신 Skill로 만들어 둡니다.

완성한 시스템의 구성

	클론 직후	챕터 7 종료
규칙	없음	21개
테스트	no tests ran	29 passed
입출력 형식	CSV · 터미널	CSV · Markdown · 터미널 · JSON
진입점	CLI	CLI · 웹 API · 웹 화면
규칙 등록	명시적 나열	자동
<code>.claude/</code>	없음	CLAUDE.md · Hook · 명령 · Skill · Subagent · 권한 설정

클론 직후부터 챕터 7을 마칠 때까지, 세세한 지시와 반복되는 설명이 줄었습니다.
사용자와 Claude Code가 공유하는 맥락이 그만큼 많아졌기 때문입니다.

확인 항목

- ☐ 다건 규칙 R017 · R019 · R020 구현과 등록 완료
- ☐ R021 픽스처와 테스트를 직접 작성
- ☐ 집계 처리 기준 세 가지를 요청에 함께 전달
- ☐ Markdown 리포트에 집계 표 포함
- ☐ 웹 API와 화면 동작 확인
- ☐ 규칙 등록을 자동 방식으로 교체
- ☐ 골든 세 세트 통과 유지
- ☐ R021 규정 문구를 `docs/경비규정.md` 에 추가
- ☐ Markdown 로더로 같은 판정 결과 확인
- ☐ 부서별 집계와 집계 테스트 작성
- ☐ `--save` 로 Markdown 파일 저장 확인
- ☐ `src/core` 가 웹 확장에서 바뀌지 않음
- ☐ 요구사항 · `CLAUDE.md` · Skill · Hook을 새 등록 방식에 맞게 수정
- ☐ 변경을 주제별로 나눠 커밋하고 PR 본문을 `outputs/pr.md` 에 작성

저장소에 남은 것은 규칙 스물한 개와 진입점 셋, 그리고 그것을 지탱하는 하네스입니다.

다음에 규칙을 하나 더 추가할 때는 파일 하나를 만드는 것으로 끝납니다.